

IT314 - Software Engineering

Lab Assignment-3 (Day-1) – Automated Pipeline for identifying FR and NFR.

You need to identify the stakeholders, elicitation techniques, identification of functional and non-functional requirements.

1. Background

Requirements Elicitation is one of the most human-dependent stages of the software engineering lifecycle it involves identifying stakeholders, understanding their goals, choosing the right technique to draw out information from them, and finally converting that information into Functional and Non-Functional Requirements.

In this assignment, you will build an **LLM-powered pipeline** that automates this elicitation process for a given system, using LangChain and prompting techniques covered in the previous lab.

2. Task

You are required to build a LangChain-based pipeline that takes the case study as input and progresses it through the following stages of requirements elicitation:

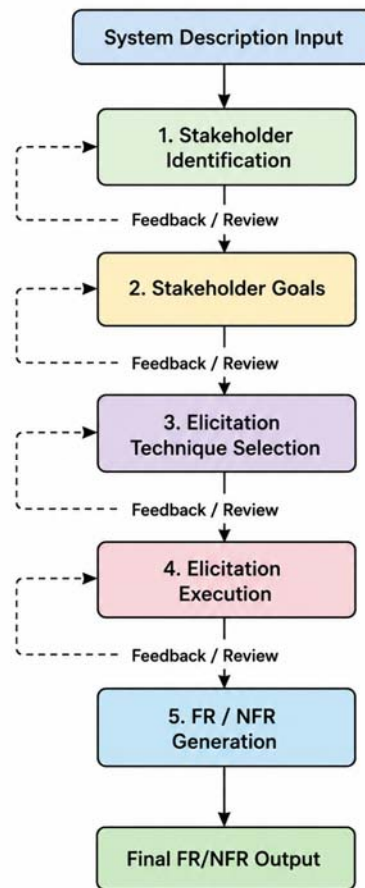
1. **Stakeholder Identification** Identify the relevant stakeholders for the given system.
2. **Stakeholder Goals** for each of the stakeholders carried forward from Stage 1, identify their goals/needs from the system.
3. **Elicitation Technique Selection** Based on the stakeholders and their goals, decide which elicitation technique(s) are appropriate (e.g., interview, questionnaire, workshop, brainstorming, observation, etc.).
4. **Elicitation Execution** Carry out the chosen technique(s) e.g., if a questionnaire is selected, generate the actual questions; if an interview is selected, decide whom to interview and draft the interview questions.
5. **FR/NFR Generation** Using everything gathered in the previous stages, generate a structured list of Functional Requirements (FRs) and Non-Functional Requirements (NFRs) for the system.

Mandatory Design Requirement Feedback at Every Stage

The pipeline **must not** run the above five stages blindly. At each stage transition, there must be **some feedback/review mechanism** that lets a review, edit, approve, or steer the output of one stage before it is carried forward into the next.

What form this feedback mechanism takes is intentionally left for you to design and justify. There is no single correct implementation think about what is practical to build and appropriate for each stage, and be ready to explain your reasoning. Hint: Human-in-the-loop (HITL), ClarifyLLM, LLM as a Judge etc., you can develop for feedback mechanism.

3. Expected Pipeline (Block Diagram)



4. Target System

Smart Campus Cafeteria / Food Court Ordering & Feedback System

Most college campuses run their cafeterias and food courts the same way they have for decades: students physically walk up to a counter, wait in a queue that grows unpredictably long around lunchtime, place an order verbally or by pointing at a menu board, and then wait again often without knowing how long the wait will actually be before their food is ready. During peak hours, this creates overcrowding near the counters, long queues that spill into walkways, and a poor experience for students who often have limited time between classes or lab sessions. Cash and card handling at the counter further slows down the process, and mistakes in orders (wrong item, missed customization, or unavailable item not communicated in time) are common because everything is handled manually under time pressure.

On the vendor's side, the cafeteria staff currently manage the entire operation menu planning, pricing, inventory of ingredients, and daily sales largely through manual registers or, at best, a basic point-of-sale system that does not talk to anything else. This makes it difficult for the vendor to know in advance how much of a particular dish is likely to be ordered on a given day, which leads to either food wastage (over-preparation) or stock-outs (under-preparation) that frustrate students. There is also no structured way for the vendor to collect feedback on food

quality, taste, or hygiene, so complaints tend to travel informally through word-of-mouth rather than reaching the vendor or administration in a form that can actually be acted upon.

The college administration, meanwhile, is expected to oversee the cafeteria's operations from a governance standpoint ensuring food pricing stays reasonable and within agreed contractual limits with the vendor, that hygiene and food-safety standards are being met, and that the cafeteria is not just running smoothly but also delivering value for the subsidized or contracted arrangement the college has with the vendor. Currently, this oversight largely depends on periodic manual inspections and student complaints reaching the administration through informal channels (word of mouth, occasional emails, or complaints raised in student council meetings), rather than through any continuous or data-driven process. For hostel students in particular, there is an added layer of complexity: many colleges tie cafeteria usage to a hostel mess plan or subscription, meaning the hostel warden's office also has a stake in how meal plans, billing, and usage patterns are tracked.

The college is now looking to modernize this entire experience by introducing a **Smart Campus Cafeteria / Food Court Ordering & Feedback System** a digital platform that would allow students to browse the menu, place and customize orders, pay digitally, and track order status in real time (thereby reducing physical queueing), while giving the cafeteria vendor tools to manage the menu, track inventory and sales trends, and forecast demand more accurately. The system is also expected to give the administration visibility into overall operations pricing compliance, order volumes, hygiene-related feedback, and student satisfaction without administration having to depend purely on informal complaints reaching them. Additionally, the system should provide a structured, easy way for students to leave feedback or ratings on specific orders or dishes, so that recurring issues (e.g., a particular dish consistently rated poorly, or repeated complaints about a specific time slot) can be surfaced and addressed rather than getting lost.

5. Deliverables

- **Working code** implementing the pipeline described above, built using LangChain and LLM API calls.
- **Prompts developed at each stage of the pipeline**
- **Output Screenshots**
- **A short report** (2–3 pages) explaining:
 - The feedback/review mechanism you designed at each stage, and why.
 - Any notable prompting or design decisions made at each stage.